Title Page

Project title	Smart Study Planner — Greedy-Based Task Scheduling System
Subject	Design and Analysis of Algorithms (DAA) Lab
Student name	Sidhanth Dahake
Roll number	146
Class	5th Semester, Section A, Department of Information Technology
College	Priyadarshini College of Engineering, Nagpur
Faculty / Guide	*[fill in: name of your DAA lab faculty]*
Academic year	*[fill in]*
GitHub repository	*[fill in: paste your repository URL after you push]*

> Items in *[square brackets]* are intentionally left blank because they are not known to the author of this template. Replace them before submission.

Abstract

Students usually have more study tasks than study hours. This project builds a small web application, the **Smart Study Planner**, that decides *which* tasks to study and *in what order*, given each task's duration, deadline and importance, and the total hours the student has available. The decision is made by a **greedy algorithm**: tasks are ranked by *importance points per hour*, accepted one by one as long as they fit in the time budget and do not break any deadline, and the accepted tasks are finally ordered by *Earliest Deadline First* (EDF). The program is written in Python with a Streamlit interface. The report explains the algorithm, proves that the schedule it returns is always valid, shows by a counter-example that it is **not always optimal** (it is a heuristic), analyses time and space complexity, and compares it with a simple first-come-first-served baseline and with an exact brute-force solution for small inputs. The implementation is covered by 59 automated tests, all of which passed in the run reported in Section 11.

1. Problem Statement and Objectives

1.1 Problem statement

Given:

- a list of n study tasks; task i has a duration d_i (hours), a deadline (a calendar date, converted to a day number) and an importance (priority) p_i from 1 (low) to 5 (high);
- the total number of hours H the student can study in all;
- the number of hours h the student studies per day (needed to convert a deadline date into a time limit).

Choose a subset of the tasks and an order for them so that

1. the sum of the chosen durations is at most H;
2. every chosen task is finished on or before its deadline;
3. the total importance of the chosen tasks is as large as possible.

Tasks that are not chosen are **deferred** (postponed), and the program must say why.

1.2 Objectives

- Implement a greedy scheduling algorithm with a clearly documented rule.
- Show selected tasks, deferred tasks, total scheduled hours, remaining hours and a day-by-day plan.
- Validate all input and handle: no tasks, invalid durations, insufficient time.
- Analyse correctness, time complexity and space complexity.
- Show honestly where the greedy method fails (limitations).
- Compare against a baseline strategy and against the exact optimum for small inputs.

2. Assumptions and Scope

- Study time is one continuous timeline that starts at the beginning of *today*; every day offers exactly h study hours. A task due on day d (0 = today) must therefore be finished before $(d + 1) \times h$ hours of study have been used. This is called the **deadline in hours**.
- A task is done without interruption by other tasks, but a long task may continue on the next day when the day's hours run out.
- A task is either done completely or deferred completely (no partial credit).
- Durations and importance values are the student's own estimates.
- The program keeps data only in the browser session. There is no database, by design.

3. System Overview

File	Purpose
`scheduler.py`	All algorithm code (greedy, baseline, brute force, validation, day plan). No Streamlit inside.
`app.py`	Streamlit screen: forms, tables, buttons, messages.
`test_scheduler.py`	28 test functions (49 test cases once parameters are expanded) for the algorithm.
`test_app_ui.py`	10 headless tests of the Streamlit screen using Streamlit's `AppTest`.

Technology: Python 3, Streamlit (web interface), pandas (installed together with Streamlit, used for tables), pytest (tests).

Screen layout: a sidebar for the time budget and demo buttons; Step 1 add a task; Step 2 task list; Step 3 results with four numbers (selected, deferred, scheduled hours, remaining hours) and five tabs (schedule, selected, deferred, greedy vs baseline, algorithm and limitations).

4. The Greedy Algorithm

4.1 Documented rule

> **Selection rule.** Rank tasks by *importance ÷ duration* (importance points gained per hour of study), highest first. Break ties by earlier deadline, then shorter duration, then name. Go through the ranked list once; **accept** a task only if (a) it still fits in the remaining hours and (b) the accepted tasks, ordered by earliest deadline, can all still meet their deadlines. Otherwise **defer** it.
>
> **Ordering rule.** Study the accepted tasks in **Earliest Deadline First** (EDF) order.

Why this rule? Importance per hour is the natural "best value for the time spent" measure, the same idea used in the fractional knapsack problem. EDF is the classical rule for meeting deadlines.

4.2 Algorithm steps

1. Compute ratio = importance ÷ duration for every task.
2. Sort the tasks by the ratio (highest first) using the tie-breaks above. Call this the *ranked list*.
3. Set `chosen = []`, `deferred = []`, `used = 0`.
4. For each task *t* in the ranked list:
    1. If *t* cannot finish by its own deadline even when studied first, defer it (*deadline too close*).
    2. Else if `used + duration(t) > H`, defer it (*not enough remaining hours*).
    3. Else if `chosen + [t]`, sorted by EDF, would make some task finish after its deadline, defer it (*would break a deadline*).
    4. Else add *t* to `chosen` and add its duration to `used`.
5. Sort `chosen` by EDF. This is the study order.
6. Report selected tasks, deferred tasks (with reasons), `used`, `H − used`, and split the study order into days of *h* hours.

4.3 Pseudocode

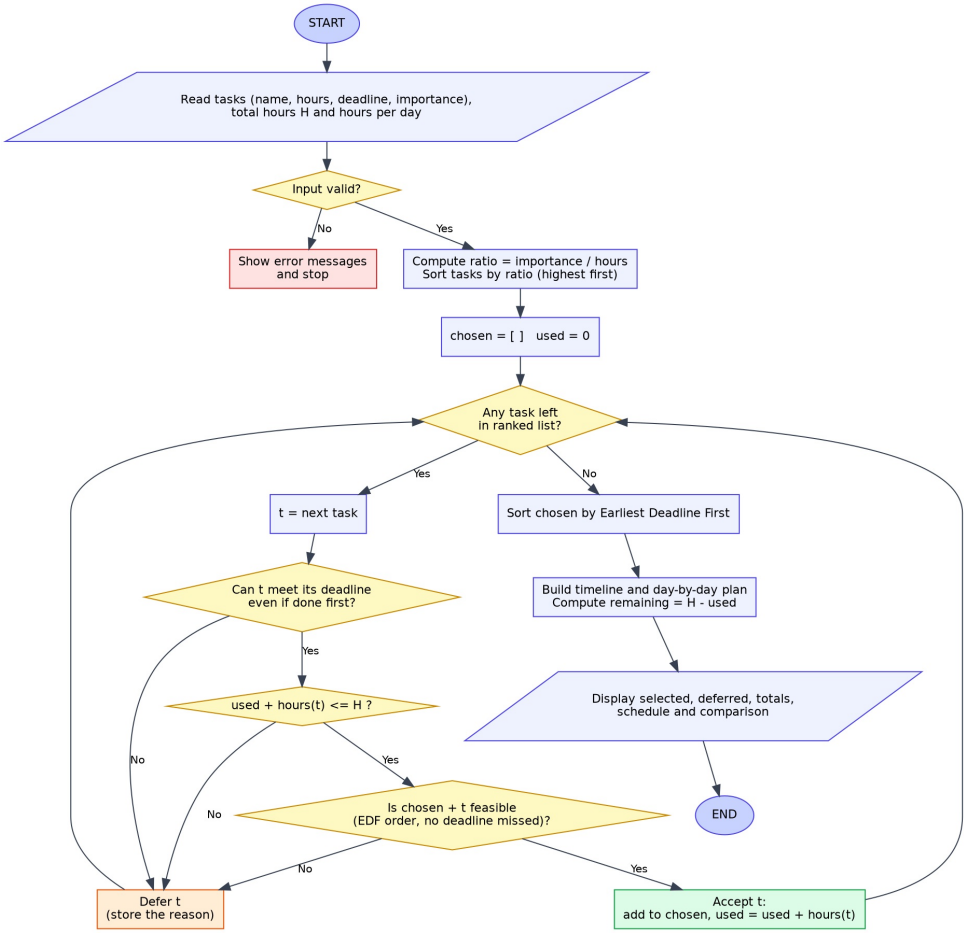
```
GREEDY-STUDY-SCHEDULE(tasks, H, h)
 1  for each task t in tasks
 2      ratio[t] <- priority[t] / duration[t]
 3  ranked   <- tasks sorted by (ratio desc, deadline asc, duration asc, name asc)
 4  chosen   <- empty list
 5  deferred <- empty list
 6  used     <- 0
 7  for each task t in ranked
 8      if duration[t] > DEADLINE-HOURS(t, h)            // cannot finish even if first
 9          add (t, "deadline too close") to deferred
10      else if used + duration[t] > H
11          add (t, "not enough hours") to deferred
12      else if not FEASIBLE( EDF( chosen + [t] ), h )
13          add (t, "would break a deadline") to deferred
14      else
15          add t to chosen
16          used <- used + duration[t]
17  schedule <- EDF(chosen)
18  return schedule, deferred, used, H - used

DEADLINE-HOURS(t, h)
 1  return (deadlineDay[t] + 1) * h

FEASIBLE(L, h)               // L is a list of tasks in the order they will be studied
 1  clock <- 0
 2  for each task t in L, in order
 3      clock <- clock + duration[t]
 4      if clock > DEADLINE-HOURS(t, h)
 5          return FALSE
 6  return TRUE

EDF(list)                    // Earliest Deadline First
 1  return list sorted by (deadlineDay asc, priority desc, duration asc, name asc)
```

5. Flowchart



Flowchart of the greedy study-scheduling algorithm

The editable source of the flowchart is `docs/flowchart.dot` (Graphviz).

6. Correctness Explanation

"Correct" needs care here. The algorithm is **always valid** (it never returns a schedule that breaks the rules) and **maximal** (nothing deferred can still be added), but it is **not guaranteed optimal** (it may not collect the most importance points). The three claims below are separated on purpose.

6.1 The schedule is always valid

Claim. The returned schedule uses at most H hours and every task in it meets its deadline.

Argument (loop invariant). Before the loop starts, `chosen` is empty, so it is feasible and `used = 0 ≤ H`. A task is added to `chosen` only in step 15, i.e. only after both tests have passed: `used + duration ≤ H` (line 10) and `chosen + [t]` in EDF order is feasible (line 12). So after every iteration `used ≤ H` and `chosen` is feasible. When the loop ends the invariant still holds. We then order `chosen` by EDF; by Lemma 1 below, a feasible set stays feasible in EDF order, so the final order has no late task.

Lemma 1 (EDF is an exact feasibility test). If *any* order of a set of tasks meets all deadlines, then the EDF order also meets them.

Proof (exchange argument). Take a feasible order that is not in EDF order. Then some adjacent pair i then j has deadline(i) > deadline(j). Let s be the time at which i starts. Swap them so j runs first.

- j now finishes at $s + d_j$, which is earlier than before, so it is still on time.
- i now finishes at $s + d_j + d_i$. This is exactly the time at which j used to finish, which was ≤ deadline(j) < deadline(i), so i is on time.
- Every other task finishes at the same time as before.

Each such swap keeps the schedule feasible and removes one out-of-order pair. Repeating (like bubble sort) ends in EDF order, which is therefore feasible. ∎

6.2 The final set is maximal

Claim. After the algorithm finishes, no deferred task can be added to the final set without breaking the hour budget or a deadline.

Argument. Suppose task t was deferred when the chosen set was $C \subseteq C_{final}$ (the chosen set only grows).

- If t was deferred because `used + duration > H`: the final `used` is ≥ the earlier `used`, so t still does not fit.
- If t was deferred because `C + [t]` was infeasible (this includes "cannot meet its own deadline"): removing tasks from a feasible set cannot make it infeasible, so a set that contains an infeasible subset is infeasible. Hence `C_final + [t]` is infeasible too. ∎

This claim is also checked by the random test `TC28` in `test_scheduler.py`.

6.3 Why the ratio rule, and why it is not always optimal

Where the rule comes from. If tasks could be split into pieces and there were no deadlines, taking the best ratio first would be optimal (the fractional knapsack result): if an optimal plan used a lower-ratio hour while a higher-ratio hour was left unused, swapping them would not decrease the total importance.

Why it fails for whole tasks. Our tasks cannot be split (0/1 choice) and have deadlines. Counter-example (budget H = 10 h, 4 h/day, all deadlines far away):

Task	Duration	Importance	Ratio
A	6 h	4	0.67
B	5 h	3	0.60
C	5 h	3	0.60

Greedy takes A first (best ratio). Then B (11 h) and C (11 h) no longer fit, so the total is **4**. The best choice is B + C = 10 h with total **6**. The program reproduces exactly this (tests `TC13` and `TC27`; the "greedy is not optimal" demo button).

Why no easy fix exists. If all deadlines are far away and h is large, the problem is the 0/1 knapsack problem, which is NP-hard. Therefore no simple greedy rule can be expected to be optimal for every input. The greedy method is a **heuristic**: fast and always valid, but without an optimality guarantee. (The random test `TC12` also confirms that greedy never scores *above* the exact optimum, as it must not.)

7. Complexity Analysis

Let n be the number of tasks, $k \le n$ the number of accepted tasks.

Step	Time
Compute ratios	O(n)
Sort by ratio	O(n log n)
Main loop: n iterations; each re-sorts the accepted tasks by EDF, O(k log k), and checks feasibility, O(k)	O(n · n log n) = O(n² log n) worst case
Final EDF sort	O(n log n)
Build timeline and day plan	O(n + D) where D is the number of days used
Total (as implemented)	**O(n² log n)**

Space: the ranked list, `chosen` and `deferred` each hold at most n tasks: **O(n)**.

Possible improvement: if the accepted tasks are kept in a sorted structure and the new task is inserted at its place (instead of re-sorting), each iteration costs O(n) and the total becomes **O(n²)**. The simple version was kept because it is easier to explain.

Other strategies in the project

Strategy	Time	Space
Baseline (first-come-first-served)	O(n) selection, O(n) timeline	O(n)
Brute force (all subsets)	O(2^n · n log n)	O(n)

Brute force is used only for $n \le 12$ (at most 4096 subsets), so the page stays fast.

8. Input Validation and Error Handling

Situation	Behaviour
No tasks	Message “No tasks yet…”; results are not computed.
Empty or over-long name, duplicate name	Error message; task not added.
Duration 0, negative, not a number, NaN, infinity, or more than 100 h	Error message; task not added.
Importance outside 1–5 or not a whole number	Error message; task not added.
Deadline before today	Error message; task not added.
Total hours ≤ 0, or hours per day ≤ 0 or > 24	Error message; schedule not computed.
Not enough time for any task	Warning “No task could be scheduled…”.
Not enough time for some tasks	Warning plus a *Deferred tasks* tab with a reason for each.

9. Baseline Comparison

The **baseline** takes tasks in the order they were entered while they fit in the hours; it ignores importance and deadlines. The app also shows the **optimal** answer found by brute force when there are 12 tasks or fewer.

Result for the *Standard demo* (H = 14 h, 4 h/day):

Strategy	Tasks	Hours used	Importance points	Tasks missing deadline	On-time points
Greedy (importance/hour + EDF)	5	14	19	0	19
Baseline (first-come-first-served)	5	14	13	2	6
Optimal (brute force)	5	14	19	0	19

Result for the *“greedy is not optimal” demo* (H = 10 h):

Strategy	Tasks	Hours used	Importance points
Greedy	1	6	4
Baseline	1	6	4
Optimal (brute force)	2	10	6

Fair reading of the comparison. The baseline’s outcome depends on the order in which tasks are typed in, and the standard demo lists the less important tasks first, so the baseline does badly there. This is an illustration, not a proof that greedy beats first-come-first-served on every input. The second demo shows an input on which greedy itself falls short of the optimum.

10. Sample Input and Output

Input (Standard demo; deadlines are “days from today”, 0 = today). Total available hours H = **14**, study hours per day = **4**.

#	Task	Duration (h)	Deadline	Importance	Ratio
1	Resume Update	1	day +7	1	1.00
2	Python Practice	2	day +6	2	1.00
3	Mock Test Revision	5	day +0	3	0.60
4	Computer Networks Notes	4	day +4	3	0.75
5	Blockchain Mini Project	5	day +5	5	1.00
6	Maths Problem Set	3	day +3	4	1.33
7	DBMS Normalization Revision	2	day +2	4	2.00
8	DAA Assignment	3	day +1	5	1.67

Ranked order (ratio, then earlier deadline): DBMS Normalization Revision (2.00), DAA Assignment (1.67), Maths Problem Set (1.33), Blockchain Mini Project (1.00, due +5), Python Practice (1.00, due +6), Resume Update (1.00, due +7), Computer Networks Notes (0.75), Mock Test Revision (0.60).

Output — summary

Selected	Deferred	Scheduled hours	Remaining hours
5	3	14	0

Selected tasks (study order, EDF)

Order	Task	Hours	Timeline hours	Finishes on	On time
1	DAA Assignment	3	0 – 3	Day 1	Yes
2	DBMS Normalization Revision	2	3 – 5	Day 2	Yes
3	Maths Problem Set	3	5 – 8	Day 2	Yes
4	Blockchain Mini Project	5	8 – 13	Day 4	Yes
5	Resume Update	1	13 – 14	Day 4	Yes

Deferred tasks

Task	Reason
Python Practice	Not enough remaining study hours
Computer Networks Notes	Not enough remaining study hours
Mock Test Revision	Deadline too close: it cannot finish in time even if started first

Day-by-day plan (4 h per day)

Day	Task	Hours
Day 1	DAA Assignment	3
Day 1	DBMS Normalization Revision	1
Day 2	DBMS Normalization Revision	1
Day 2	Maths Problem Set	3
Day 3	Blockchain Mini Project	4
Day 4	Blockchain Mini Project	1
Day 4	Resume Update	1

These values were produced by running `scheduler.py` on this input (also asserted in test `TC26`). The app shows calendar dates instead of “Day n”, because “today” is taken from the computer’s clock.

11. Test Cases and Results

The complete table with inputs, expected results and actual results is in `TEST_CASES.md`. Summary of the automated run:

Item	Result
Command	`python -m pytest -v`
Environment	Python 3.12.3, pytest 9.1.1, Streamlit 1.65.0 (Linux)
Algorithm tests (`test_scheduler.py`)	49 passed
Screen tests (`test_app_ui.py`)	10 passed
Total	**59 passed, 0 failed**

Selected cases:

ID	What is tested	Expected	Actual
TC01	No tasks, H = 10	nothing selected, remaining 10	as expected
TC03	One 6 h task, H = 5	deferred: not enough hours	as expected
TC05	Ratio 2.0 task vs ratio 1.25 task, room for one	the ratio-2.0 task is chosen	as expected
TC08	5 h task due today with 4 h/day	deferred: deadline too close	as expected
TC09	1 h task after a 4 h task, both due today	second task deferred (would break deadline)	as expected
TC13	Counter-example of Section 6.3	greedy 4 points, optimal 6 points	as expected
TC16	Duration 0, −2, NaN, “abc”, 100.5	validation error	as expected
TC26	Standard demo	5 selected, 19 points, 14 h used	as expected
UI08	Sample tasks with H = 0.5	0 selected, warning shown	as expected

Important: the screen tests run Streamlit’s headless test harness, and a real `streamlit run app.py` start-up returned HTTP 200 on its health and home endpoints. This is *not* a substitute for opening the app in a browser. Browser checks are listed in `TEST_CASES.md` (manual part) and in `SCREENSHOT_CHECKLIST.md` for you to perform and photograph.

12. Limitations

- **Heuristic, not optimal.** Greedy scheduling does not guarantee the globally optimal schedule for every combination of deadlines and priorities (Section 6.3).
- It never reconsiders earlier decisions, so one early acceptance can block a better combination.
- The model assumes a fixed number of study hours every day, starting from the beginning of today. It does not know that part of today may already be over.
- Tasks are all-or-nothing; they cannot be partly done or interrupted for another task.
- Importance is a 1–5 integer, so it is coarse; the ratio rule favours short tasks.
- Durations are estimates entered by the user; wrong estimates give a wrong plan.
- Data lives in the browser session only; refreshing the page clears it.
- The exact optimum is computed only for 12 tasks or fewer.

13. Future Scope

- Use dynamic programming (a knapsack table) when durations are whole hours, to get the exact optimum in O(n × H).
- Add recurring tasks, breaks and different hours for each day.
- Save and load tasks from a file.

- Let the user choose between several strategies.

14. Conclusion

The Smart Study Planner shows a complete greedy-algorithm workflow: a clear rule, a loop invariant that proves the output is always valid, an exchange-argument lemma for EDF, a complexity analysis, a measured comparison with a baseline, and an honest counter-example showing the limits of the method. It is simple enough to explain line by line while still being a usable tool.

15. Output Screenshots

Insert the screenshots listed in `SCREENSHOT_CHECKLIST.md` here, each with a figure number and caption. This copy of the report does not contain screenshots, because they must be taken from your own running copy of the application.

[Figure 1: ...] [Figure 2: ...] [Figure 3: ...]

16. References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, MIT Press — chapters on greedy algorithms, activity selection, the knapsack problems and NP-completeness.
2. J. Kleinberg, É. Tardos, *Algorithm Design*, Pearson — scheduling to minimise lateness (Earliest Deadline First, exchange argument).
3. Streamlit documentation: https://docs.streamlit.io
4. Python documentation: https://docs.python.org/3/
5. pytest documentation: https://docs.pytest.org